

Prolog – Recursion

Recursion

Recursion

Recursion

Recursion

Multi-Agent Systems
Practical 3

Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].
```

Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [likes, bitterballen].
```

Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [likes, bitterballen].
```

```
?- [H, _ | T] = ['frans bauer', likes, bitterballen].
```

Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [likes, bitterballen].
```

```
?- [H, _ | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [bitterballen].
```

Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [likes, bitterballen].
```

```
?- [H, _ | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [bitterballen].
```

```
?- [_ , H | T] = ['frans bauer', likes, bitterballen].
```

Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [likes, bitterballen].
```

```
?- [H, _ | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [bitterballen].
```

```
?- [_ , H | T] = ['frans bauer', likes, bitterballen].  
H = likes,  
T = [bitterballen].
```

Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [likes, bitterballen].
```

```
?- [H, _ | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [bitterballen].
```

```
?- [_ , H | T] = ['frans bauer', likes, bitterballen].  
H = likes,  
T = [bitterballen].
```

```
?- [H | T, _] = ['frans bauer', likes, bitterballen].
```


Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [likes, bitterballen].
```

```
?- [H, _ | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [bitterballen].
```

```
?- [_ , H | T] = ['frans bauer', likes, bitterballen].  
H = likes,  
T = [bitterballen].
```

```
?- [H | T, _] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = likes.
```

Lists - Recap

A Prolog **list** has a **Head** and a **Tail**...

```
?- [H | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [likes, bitterballen].
```

```
?- [H, _ | T] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = [bitterballen].
```

```
?- [_ , H | T] = ['frans bauer', likes, bitterballen].  
H = likes,  
T = [bitterballen].
```

```
?- [H | T, _] = ['frans bauer', likes, bitterballen].  
H = 'frans bauer',  
T = likes.
```



Lists - Recap

What about the empty **list**?

```
?- is_list([]).  
true.
```

```
?- [H | T] = [].  
false.
```

```
?- [H | _] = [].  
false.
```

This will be important later...

How to find...

The length of a **list** with **helper** predicates?

List:

```
[1, 2, 3, 4, 67, 9000]
```

```
?-
```

How to find...

The length of a **list** with **helper** predicates?

```
?- length([1, 2, 3, 4, 67, 9000], L).  
L = 5.
```

List:

```
[1, 2, 3, 4, 67, 9000]
```

length/2

How to find...

The length of a **list** with **helper** predicates?

```
?- length([1, 2, 3, 4, 67, 9000], L).  
L = 5.
```

List:

```
[1, 2, 3, 4, 67, 9000]
```

length/2

The maximum value of a **list** with **built-in** predicates?

```
?-
```

How to find...

List:

```
[1, 2, 3, 4, 67, 9000]
```

The length of a **list** with **helper** predicates?

```
?- length([1, 2, 3, 4, 67, 9000], L).  
L = 5.
```

length/2

The maximum value of a **list** with **built-in** predicates?

```
?- max_list([1, 2, 3, 4, 67, 9000], Max).  
Max = 9000.
```

max_list/2

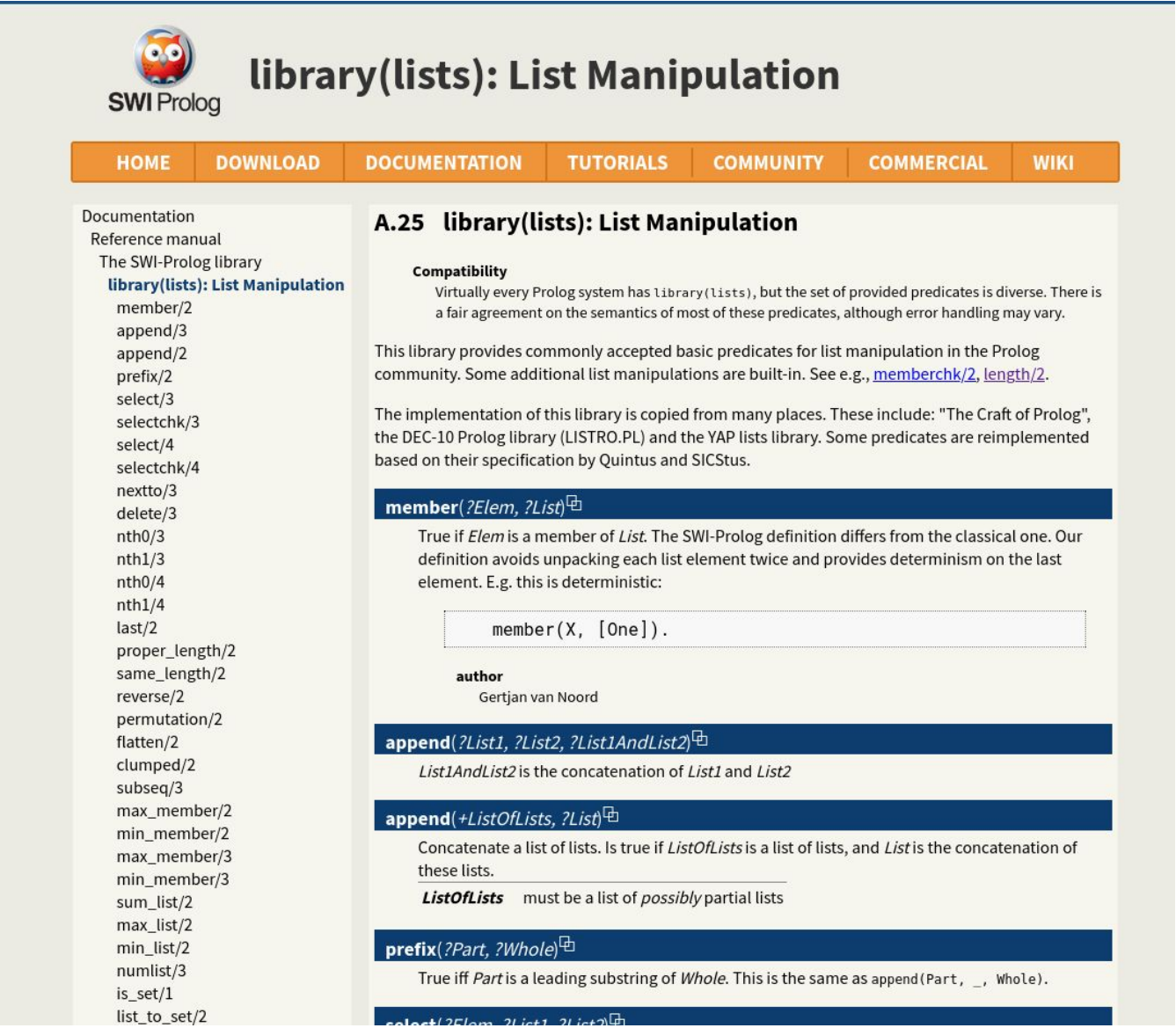
Read the docs!

Some more **helper** predicates for **lists**:

```
?- append([a,b], [c], X).  
X = [a,b,c].
```

```
?- select(2, [1, 2, 3], R).  
R = [1, 3];
```

```
?- is_list('I am a list').  
False.
```



The screenshot shows the SWI-Prolog website's documentation for the `library(lists)`. The page has a navigation bar with links: HOME, DOWNLOAD, DOCUMENTATION, TUTORIALS, COMMUNITY, COMMERCIAL, and WIKI. The left sidebar lists various predicates under the heading "library(lists): List Manipulation", including `member/2`, `append/3`, `prefix/2`, `select/3`, `selectchk/3`, `selectchk/4`, `nextto/3`, `delete/3`, `nth0/3`, `nth1/3`, `nth0/4`, `nth1/4`, `last/2`, `proper_length/2`, `same_length/2`, `reverse/2`, `permutation/2`, `flatten/2`, `clumped/2`, `subseq/3`, `max_member/2`, `min_member/2`, `max_member/3`, `min_member/3`, `sum_list/2`, `max_list/2`, `min_list/2`, `numlist/3`, `is_set/1`, and `list_to_set/2`. The main content area is titled "A.25 library(lists): List Manipulation" and includes a "Compatibility" section, a paragraph about the library's purpose, and detailed documentation for the `member`, `append`, and `prefix` predicates, each with its signature and a brief description.

library(lists): List Manipulation

Compatibility
Virtually every Prolog system has `library(lists)`, but the set of provided predicates is diverse. There is a fair agreement on the semantics of most of these predicates, although error handling may vary.

This library provides commonly accepted basic predicates for list manipulation in the Prolog community. Some additional list manipulations are built-in. See e.g., [memberchk/2](#), [length/2](#).

The implementation of this library is copied from many places. These include: "The Craft of Prolog", the DEC-10 Prolog library (LISTRO.PL) and the YAP lists library. Some predicates are reimplemented based on their specification by Quintus and SICStus.

member(?Elem, ?List)
True if *Elem* is a member of *List*. The SWI-Prolog definition differs from the classical one. Our definition avoids unpacking each list element twice and provides determinism on the last element. E.g. this is deterministic:

```
member(X, [One]).
```

author
Gertjan van Noord

append(?List1, ?List2, ?ListAndList2)
ListAndList2 is the concatenation of *List1* and *List2*

append(+ListOfLists, ?List)
Concatenate a list of lists. Is true if *ListOfLists* is a list of lists, and *List* is the concatenation of these lists.
ListOfLists must be a list of *possibly partial* lists

prefix(?Part, ?Whole)
True iff *Part* is a leading substring of *Whole*. This is the same as `append(Part, _, Whole)`.

<https://www.swi-prolog.org/pldoc/man?section=lists>

Recursion

Traversing through **lists**

{ Knowledge Base }

```
in_list(X, [X|T]).
```

What happens when we pose this query?

```
?- in_list(yolanda, [yolanda, trudy, vincent, jules]).
```

Recursion

Traversing through **lists**

{ Knowledge Base }

```
in_list(X, [X|T]).
```

What happens when we pose this query?

```
?- in_list(yolanda, [yolanda, trudy, vincent, jules]).  
true.
```

```
% x = x, yolanda = yolanda
```

Recursion

Traversing through **lists**

{ Knowledge Base }

```
in_list(X, [X|T]).
```

What happens when we pose this query?

```
?- in_list(yolanda, [yolanda, trudy, vincent, jules]).  
true.
```

```
% x = x, yolanda = yolanda
```

And this query?

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

Recursion

Traversing through **lists**

{ Knowledge Base }

```
in_list(X, [X|T]).
```

What happens when we pose this query?

```
?- in_list(yolanda, [yolanda, trudy, vincent, jules]).  
true.
```

```
% x = x, yolanda = yolanda
```

And this query?

```
?- in_list(yolanda, [vincent, trudy, yolanda]).  
false.
```

```
% X \= X, yolanda \= vincent
```

Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }

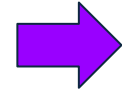
```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x

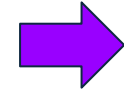
x

yolanda = vincent

Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

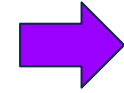
x **x**
yolanda = vincent

T = ?

Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x

x

yolanda = vincent

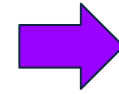
T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```


Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x

x

yolanda = vincent

T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```

x

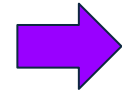
x

yolanda = ?

Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x

x

yolanda = vincent

T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```

x

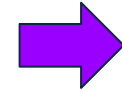
x

yolanda = trudy

Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x

x

yolanda = vincent

T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```

x

x

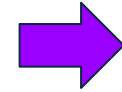
yolanda = trudy

T = ?

Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x

x

yolanda = vincent

T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```

x

x

yolanda = trudy

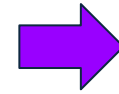
T = [yolanda]

```
in_list(yolanda, [yolanda]).
```

Recursion

Let's add a recursive rule and perform the same query...

{ Knowledge Base }



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x

x

yolanda = vincent

T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```

x

x

yolanda = trudy

T = [yolanda]

```
in_list(yolanda, [yolanda]).
```

x

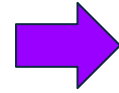
x

yolanda = yolanda

true

Recursion

Let's add a recursive rule and perform the same query...



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

{ Knowledge Base }

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x **x**
yolanda = vincent

T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```

x **x**
yolanda = trudy

T = [yolanda]

```
in_list(yolanda, [yolanda]).
```

x **x**
yolanda = yolanda

true

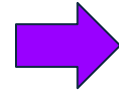
T = ?

But...

will it terminate?

Recursion

Let's add a recursive rule and perform the same query...



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

{ Knowledge Base }

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x **x**
yolanda = vincent

T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```

x **x**
yolanda = trudy

T = [yolanda]

```
in_list(yolanda, [yolanda]).
```

x **x**
yolanda = yolanda

true

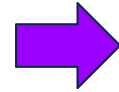
T = []

But...

will it terminate?

Recursion

Let's add a recursive rule and perform the same query...



```
in_list(X, [X|T]).  
in_list(X, [H|T]) :- in_list(X, T).
```

{ Knowledge Base }

```
?- in_list(yolanda, [vincent, trudy, yolanda]).
```

x **x**
yolanda = vincent

T = [trudy, yolanda]

```
in_list(yolanda, [trudy, yolanda]).
```

x **x**
yolanda = trudy

T = [yolanda]

```
in_list(yolanda, [yolanda]).
```

x **x**
yolanda = yolanda

true

T = []

[**x** | **T**] = []

does not
unify

But...

will it terminate?

{ Knowledge Base }

Base clause

```
in_list(X, [X|T]).
```

Recursive clause

```
in_list(X, [H|T]) :- in_list(X, T).
```

Tips and tricks

- Practice makes perfect!
 - Try to recognize the different patterns of how the example problems are solved and map them onto a new problem.
- First formulate a solution in your own words / pseudo-code before trying to create the Prolog predicates.
- List recursion:
 - Typically steps through a list one way or another
 - `[H | T]`
 - `[a,b,c,d]`
 - `[a | [b, c, d]]`, `[a, b | [c, d]]`, `[a, b, c | [d]]`, or `[a, b, c, d | []]`
 - `[a | [b | [c | [d]]]]`
 - Base clause: what is the most logical end-point of stepping through a list.
 - `length([],0).`
 - `max([X],X).`
 - Recursive clause: which mechanism do you need?
 - List decomposition? (e.g. `append/3`)
 - Element comparison? (e.g. `del/3`, `allSame/1`)
 - Counter? (e.g. `length/2`)
- Do you need one or more base clauses or recursive clauses?
- Do you need helper predicates?

Exercise:

flatten **lists**

```
[a, [b, c], [D, [5, 6]]]
```



```
[a, b, c, D, 5, 6]
```

flatten_list/2 flattens a nested list, removing all nested lists and combining elements into a single list. You can use append/3.

Exercise:

flatten **lists**

```
[a, [b, c], [D, [5, 6]]]
```



```
[a, b, c, D, 5, 6]
```

What are the three conditions?

Exercise:

flatten **lists**

[a, [b, c], [D, [5, 6]]]

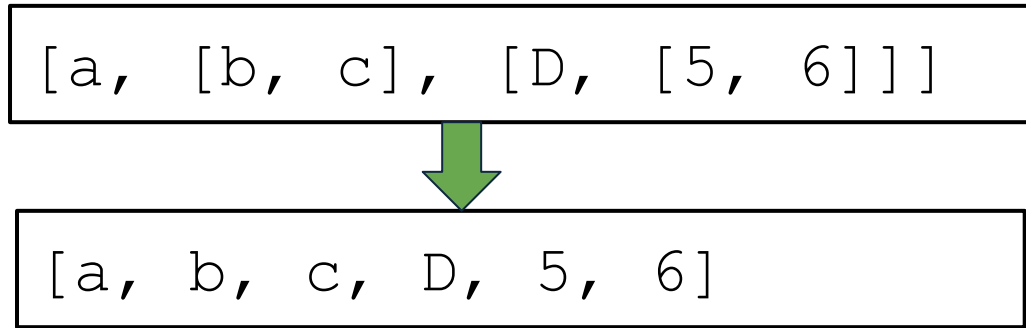


[a, b, c, D, 5, 6]

1. an empty list flattens to an empty list.
2. if the head is a list, flatten it and concatenate it with the flattened tail.
3. if the head is not a list, add it to the flattened tail.

Exercise:

flatten **lists**



1. an empty list flattens to an empty list.
2. if the head is a list, flatten it and concatenate it with the flattened tail.
3. if the head is not a list, add it to the flattened tail.

```
% Base clause  
flatten_list([], []).
```

```
% Recursive clause 1:
```

```
flatten_list([Head|Tail], FlatList) :-  
    is_list(Head),  
    flatten_list(Head, FlatHead),  
    flatten_list(Tail, FlatTail),  
    append(FlatHead, FlatTail, FlatList).
```

```
% Recursive clause 2:
```

```
flatten_list([Head|Tail], [Head|FlatTail]) :-  
    \+ is_list(Head),  
    flatten_list(Tail, FlatTail).
```

Example: duplicating elements in list

Goal

`duplicateList(L, D)` is true iff each element in `L` is duplicated in `D`.

?- `duplicateList([a, b, c], D)`.

`D = [a, a, b, b, c, c]`.

?- `duplicateList([1, 2, 3, 4], D)`.

`D = [1, 1, 2, 2, 3, 3, 4, 4]`.

Solution: duplicating elements in list

Goal

?- duplicateList([a, b, c], Duplicated).
Duplicated = [a, a, b, b, c, c].

?- duplicateList([1, 2, 3, 4], Duplicated).
Duplicated = [1, 1, 2, 2, 3, 3, 4, 4].

% Base clause: an empty list duplicates to an empty list.
duplicateList([], []).

% Recursive clause: duplicate the head of the list, and then recursively duplicate the tail.
duplicateList([Head|Tail], [DupHead1, DupHead2 | DupTail]) :-
 duplicateElement(Head, [DupHead1, DupHead2]), % Duplicate the head
 duplicateList(Tail, DupTail). % Recursively duplicate the tail.

% Helper predicate to duplicate a single element.
duplicateElement(Element, [Element, Element]).

Exercise: count vowels in a list

```
?- nr_vowel([],X).
```

```
X = 0
```

```
?- nr_vowel([a,r,e,d,i],X).
```

```
X = 3
```

```
?- nr_vowel([m,r],X).
```

```
X = 0
```

```
?- nr_vowel([s,e,e,d],X).
```

```
X = 2
```

Solution: count vowels in a list

```
% helper predicate
vowel(X):- member(X,[a,e,i,o,u]).

% base case
nr_vowel([],0).

% recursive clauses
nr_vowel([X|T],N) :-
    vowel(X), nr_vowel(T,N1), N is N1+1.
nr_vowel([X|T],N):-
    \+ vowel(X), nr_vowel(T,N).
```